Q1.

```
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) {
        printf("*");
    }
}
```

What is the time complexity of this code in terms of n?

Q2.

```
for (int i = 0; i < n; i++) { /* constant amount of work */ }
for (int j = 0; j < n; j++) { /* constant amount of work */ }
```

These two loops run one after another (not nested). What is the overall time complexity?

A. $O(n^2)$, because there are two loops over n

B. O(n), since the loops' costs add rather than multiply

C. O(log n)

D. O(n) as the first loop is never executed

Q3.

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + f(n - 1);
}
```

Counting the initial call itself, exactly how many total calls to f occur when we evaluate f(4)?

Q4. Starting from an empty stack, the following operations are performed in order: push(4), push(9), pop(), push(2), push(7), pop(), pop(), push(1)

What value is returned by the **second** pop() call in this sequence?

A. 9

B. 2
C. 7
D. Results in underflow (attempting to pop from empty stack)

Q5. Using the standard stack-based bracket-matching algorithm, consider the string: {[(])}

Is this string balanced? If not, at which character (by position, position count starting with 1) is the mismatch first detected?

Q6.

```
#define MAX 4
int arr[MAX], top = -1;


void push(int x) {
    if (top == MAX - 1) { printf("Overflow\n"); return; }
    arr[++top] = x;
}
```

If we call push(10), push(20), push(30), push(40), push(50) in that order: How many times does "Overflow" get printed, and on which call?

Q7. A circular queue uses array capacity `CAPACITY = 6`, with a separate count variable tracking size (so no slot is wasted). The rules are:

- `enqueue(x)`: store `x` at `arr[rear]`, then set `rear = (rear + 1) % CAPACITY`.
- `dequeue()`: read `arr[front]`, then set `front = (front + 1) % CAPACITY`.

Starting with `front = rear = 0` (empty queue): enqueue 4 elements, then dequeue 2 elements, then enqueue 3 more elements. What is the final value of rear?

Q8. Starting from an empty queue, perform: enqueue(A), enqueue(B), dequeue(), enqueue(C), enqueue(D), dequeue(), dequeue(), enqueue(E). What is the queue's content, listed from front to rear, after all operations?

Q9.

```
struct Node { int data; struct Node* next; };
struct Node* head = NULL;
```

```
    void insertFront(int x) {
        struct Node* n = malloc(sizeof(struct Node));
        n->data = x;
        n->next = head;
        head = n;
}
```

If we call insertFront(3), insertFront(7), insertFront(1), insertFront(9) in that order, what does the list look like when printed from head to tail?

Q10. Consider a singly linked list with only a head pointer (no tail pointer, no stored size). (a) What is the time complexity of inserting a new node at the **front** of the list? (b) What is the time complexity of inserting a new node at the **end** of the list?

Q11. An algorithm with time complexity $O(n^2)$ takes 4 milliseconds to run on an input of size $n = 10$. Assuming the constant factor stays the same and dominates the running time, approximately how long will it take to run on an input of size $n = 100$?

Q12. A stack has a maximum capacity of 5 elements and currently holds 3 elements. Without any pops in between, we attempt 4 push operations, one after another. How many of these 4 pushes result in an overflow?